

面向多轮交互式长编程任务的记忆自优化

方向 B：多轮交互中的高置信优化反馈

方向 E：记忆生命周期——更新、过时与遗忘

方案介绍 + 测试结论报告

基于 MemoryCore v2.0.0-beta.1 的可演化项目记忆、编码代理执行闭环与公开数据验证

日期	2026-09-14
分支	delivery/topic3-be-project-memory-v1
实现快照	84ba04dee30bd72d7e9f9e57365c5015e7cd5e6d
代码评审	GitHub Pull Request #4

报告摘要

长程编码任务中的异常类型、默认参数、兼容边界和目录例外，常常只存在于早期用户纠正或 review 中。代理忘记这些决定会重复犯错；无差别注入全部历史又会引入过时信息、作用域误用和额外成本。本课题选择 原题 B、E 两个方向，在 MemoryCore v2.0.0-beta.1 上实现旁路式项目记忆：B 保存可追溯、可限定、可演化的反馈，E 将其接入编码、检查、恢复和审阅闭环。

最终系统默认保留用户原话，不调用模型总结；结构化约束必须具有唯一原文引用、合法作用域和明确前任，并按项目 revision 持久复用。编码侧保存 prompt、context、模型原始事件、usage、checker、工作区差异 和持久化状态。公开验证覆盖 MemoryCode 360 段对话与 4,182 个查询、8 个公开项目家族与 16 个有效 顺序任务，以及 CUPID、ValidMem、Trigger、STALE 的补充组件实验。

核心结果是：公开项目家族按独立首任务计 2 胜 0 负 6 平，严重回归 0；MemoryCode 全部 10 个 更新 dialogue 中，Codex 使用完整原话为 7/10，无历史为 0/10；机制修正后的独立确认集由 9/10 提升至 10/10。工程侧三套主测试 260/260 通过；模型前 bridge 由 4 个进程收敛至 1 个，20 组三臂逐字等价验证中 mean/p95 固定开销下降 46.1%/46.5%。

现有证据支持“项目历史能在维护者选择无法由当前源码唯一推出时改善编码结果”，不支持普遍优于所有 编码代理或开源记忆系统。报告保留负结果、失效矩阵和未完成边界，不用不完整调用补数。

2-0-6

公开项目家族
胜-负-平

7/10

Codex 使用更新历史
无历史 0/10

260

主工程测试
全部通过

-46.1%

模型前 bridge
平均固定开销

目录

0. 原始课题描述与本次选题	4	3.4 系统边界	7
0.1 方向 B 原文	4	4. 实现与测试代码清单	8
0.2 方向 E 原文	4	4.1 主要实现	8
0.3 原目标、非目标与必交付	4	4.2 主要测试	8
0.4 B+E 闭环	5	5. 测试方法	8
1. 完成的工作与主要贡献	5	5.1 工程验证	8
1.1 最终上交材料	5	5.2 效果验证	9
1.2 设计创新与技术贡献	5	6. 测试结果	9
1.3 可核对的工作量	6	6.1 最终工程结果	9
2. 问题定义与目标	6	6.2 受控公开仓库编码任务	9
3. 方案设计	6	6.3 公共数据集与组件研究	10
3.1 B：有来源、有范围、可演化的反馈记忆	6	6.4 结论的适用范围	11
3.2 E：从模型调用到代码检查的证据闭环	7	7. 复现工程测试	11
3.3 正确性约束下的运行时优化	7	8. 最终结论	11

9. 详细量化附录	12	9.8 AI infra 最终结果	15
9.1 MemoryCode 数据与全量检索	12	10. 实现细节与代码工作量	16
9.2 MemoryCode 24-dialogue、72-call 三臂生成	13	10.1 核心数据合同	16
9.3 Codex 更新层与 focus	13	10.2 执行与证据顺序	16
9.4 CUPID、ValidMem、Trigger、STALE	14	10.3 代码规模	16
9.5 公开项目逐矩阵结果	14	11. 结论边界与最终判断	16
9.6 失效矩阵记录	15	12. 复现与移植	17
9.7 工程、fallback 与恢复	15	13. 参考资料	17

0. 原始课题描述与本次选题

以下引用来自活动方原始任务书，仅调整排版，不改变内容。

题目三：面向多轮交互式长编程任务的记忆自优化

开源 MemoryCore（建议 tag v2.0.0-beta.1）提供分层记忆（L0 对话 / L1 原子记忆 / L2 场景 / L3 画像）、HTTP Gateway，以及检索、抽取、注入等基座能力。本地检索基座通常包括 FTS5、SQLite Embedding / 向量检索与多路融合（如 RRF）；抽取、写入、注入预算、maxResults / strategy 等也多为静态配置。

本课题的应用场景是：多轮交互式长编程任务中的记忆自优化。公开可用的、真正对应这一场景的评测集目前几乎没有；活动方的真实用户数据也不能外发。因此本题不要求在公开编程 Agent 集上给出最终业务结论，而要求先做调研，在公开长对话任务上完成可复现实验，并以可审阅 PR 和可替换适配层交付，使方法后续可接入内部多轮编程数据而不用重写主逻辑。

0.1 方向 B 原文

方向 B. 多轮交互中的高置信优化反馈

自优化需要监督信号。多轮对话里，显式“你忘了 / 记错了 / 请记住”一类反馈往往精确但极稀；更常见的是当轮纠错、重述约束、撤销重做、任务失败等弱信号，直接拿来当监督又容易假阳性。

本题希望探索：在多轮交互中，如何获得能驱动记忆策略更新的反馈，并使置信度可估计、成本可接受。公开长对话上验证信号与判定方法；同样要求代码能接到内部数据。

0.2 方向 E 原文

方向 E. 记忆生命周期：更新、过时与遗忘

长程任务中约束会变（方案推翻、接口更名、环境切换）。做有上限的更新 / 失效 / 遗忘策略，避免过时记忆在后续轮次被当作事实注入。须有可复现的过时记忆用例，并报告误删代价。

0.3 原题目目标、非目标与必交付

原题把“自优化”限定为：根据评测或受控反馈更新策略参数、辅助结构或决策规则；不是一次性静态配置，也不是重写 FTS5、向量引擎或抽取模型。能力必须可关闭，读失败、超时和结构损坏时必须硬回退。公开长对话只用于方法验证，不能写成真实编程业务结论；适配层应允许后续换成内部多轮编程数据。

序号	原题必交付	本次对应成果
1	调研报告 + 设计说明（问题、相关工作、方法、协议、演化边界、fallback、移植说明）	本报告第 1-3、7-8 章及移植说明
2	公开长对话 Eval Runner 与结构化结果（含基座基线）	MemoryCode runner、JSON 结果与第 6 章
3	所选方向实现及与基座对比（或方法验证）	ProjectMemory、项目代理、公开项目与组件实验
4	关闭开关与强制失败回退验证	runtime contract、状态机与恢复测试
5	可审阅 PR：数据适配层与内部移植说明	PR #4、PORTING.md、源码提交包

0.4 B+E 闭环



本实现不把普通任务失败自动解释为记忆错误，也不让工具输出直接生成规范性约束。低置信信号最多用于 候选提名；只有用户原文、合法来源和明确范围通过校验后，才允许进入可演化当前视图。

1. 完成的工作与主要贡献

1.1 最终上交材料

五项验收要求与导师三项上交物的逐项入口见 `B_E_DELIVERY_INDEX_2026-09-14.md`。

要求	本次交付	状态
方案实现代码	TypeScript 项目记忆状态机、SQLite 接口、Python 执行宿主、Codex／Claude 后端、差异捕获、安装 CLI	完成
测试代码	TypeScript 单元／集成测试、Python 宿主／后端／差异测试、公共评测器测试、安装包 smoke、GitHub Actions	完成
方案介绍 + 测试结论报告	本文；给出问题、设计、实验协议、量化结果、复现方式和适用范围	完成

代码可读性专项复盘见 `B_E_CODE_READABILITY_REVIEW_2026-09-14.md`。CLI 的用户与维护说明见 `scripts/project-agent/README.md`。本文是给导师提交时应优先阅读的 单一总报告，前两份作为实现附录。

1.2 设计创新与技术贡献

- 把反馈记忆从文本缓存提升为可演化状态。我为观察、约束和版本关系设计了确定性标识，加入 owner／project 隔离、路径与动作范围、显式替代链和撤销语义。原始证据始终保留，当前规则可以更新，两者不再混为一层。
- 把 B 和 E 接成可审计的编码闭环。记忆不只参与提示构造，还与 Codex／Claude Code 执行、独立 checker、原始事件、usage、工作区 diff 和中断恢复关联。这样能够区分“检索到了”“模型用了”和“代码最终通过”三个不同问题。

3. 建立针对编码记忆的严格配对评测。评测以独立项目家族为统计单位，冻结 base、任务、checker 和 arm 隔离；未来 Git 引用、跨 arm 读取、联网搜索和 checker 语义漏洞都会使整组实验失效。五轮有问题的矩阵被完整保留并排除，最终只采用通过近错实现预检的 v4 和 v10。
4. 在保持语义等价的条件下优化运行时。编译结果按 revision 复用，SQLite bridge 纳入正式构建，`prepareRun` 在同一进程内完成快照读取、范围选择和任务写入。优化前后不仅比较时间，还逐字比较注入上下文和最终数据库快照，避免用性能改动悄悄改变记忆行为。

1.3 可核对的工作量

工作面	完成规模
产品实现与交付	PR 相对研究基线覆盖 41 个文件；包含状态机、SQLite 适配、双后端宿主、差异捕获、安装 CLI、构建与 CI
公开项目实验	8 个独立项目家族、16 个顺序编码任务、2 套有效矩阵；另对 v5-v9 五套失效矩阵逐一定位并保留证据
MemoryCode 研究	全量覆盖 360 段对话、4,182 个检索查询；冻结生成集 24 段对话、72 次调用；三组关键 Codex 实验为 20+30+30 次有效调用
工程测试	三套主测试共 260 项：MemoryCore 204、项目代理 25、公共评测 31；另有最低 Node、构建、安装包和尺寸门禁
运行时验证	20 组三臂配对，逐组比较上下文、数据库终态和时延；安装包在空目录完成真实安装与读写 smoke

2. 问题定义与目标

普通编码代理通常能读取当前请求和仓库，但许多真正影响实现的约定只存在于早先的用户纠正、review 意见 或例外说明中。例如，源码可以同时容纳两种异常类型，真正应采用哪一种只能由维护者历史决定。直接注入 全部历史会增加成本，也可能把旧要求错误应用到新目录；只保留自动摘要，又容易丢失“何时适用”和“为什么这样做”。因此，我把问题拆成五个可验证的问题：

1. 哪条反馈应保存，出处是什么；
2. 它对哪个项目、路径和动作生效；
3. 新要求出现后，旧要求如何保留但不再作为当前规则；
4. 编码结果是否真的通过独立行为检查；
5. 失败、中断或读取异常时，系统是否留下可审计证据并安全退回。

这里的效果指标不是“保存了多少条记忆”，也不是单独一次 checker 是否通过，而是同一任务、同一模型下有记忆与无记忆的配对行为差异，同时检查同主题控制是否回归、上下文和模型成本是否增加，以及完整证据 能否由第三方复核。

3. 方案设计

3.1 B：有来源、有范围、可演化的反馈记忆

`remember` 默认保存完整用户原话，不调用模型。观察与实验性约束都有确定性 ID；任何结构化约束都必须 引用真实存在的原文片段。这样做的目的，是把“证据是什么”和“系统怎样解释证据”分开：解释可以被修正，原始证据不会被摘要覆盖。

记忆的适用范围由 owner、project、动作和路径共同决定。非规范路径、未知动作或非法预算会在读取记忆和 调用模型之前被拒绝，不会因为异常而退回到范围更宽的历史。新规则只有明确指向同范围前任时才建立 `supersedes` 关系；旧证据仍可审计，但不会重新成为当前规则，必要时还可以通过 `retract` 显式撤销。

结构化编译通过 `remember --compile` 单独开启，每次最多提议四条带引用约束。来源、容量或格式校验失败时，系统只保留原话，不写入半成品规则。被接受的编译结果随项目 revision 持久化；revision 不变时，后续编码直接复用，不再重复付出编译调用。系统同时保留 `scoped`、`raw` 和 `off` 三种模式，分别用于范围化记忆、完整原话和无记忆基线。

3.2 E：从模型调用到代码检查的证据闭环

运行链路保持单向，每一层只承担一种职责：

```
memory-agent
-> project_agent.py (命令编排、持久化回执、恢复)
-> backend.py (Codex/Claude CLI、事件流和进程生命周期)
-> store.ts -> ProjectMemory (SQLite、范围、版本与谱系)
-> changes.py (相对 HEAD 的可审阅差异)
-> 独立 checker (行为结果，不充当用户反馈)
```

每次 `run` 都先保存任务，再调用编码后端，随后记录 agent 完成状态并执行 argv 形式的 checker。提示、记忆上下文、原始 backend 事件、usage、模型输出、检查结果和 `changes.diff` 分开落盘，审阅者可以沿一次 run 还原完整过程。超时或 Ctrl-C 会终止该进程组并留下 partial evidence。若编码已经完成、检查阶段却中断，`check-run` 可以只补跑 checker，不重复调用模型，也不新增记忆；对于人工接管的未完成 diff，`--allow-incomplete` 会保留 `completion_confirmed=false`，避免把人工确认包装成模型完成。

读取失败时，编码任务退回当前请求并记录原因；如果读取已经成功、任务写回随后失败，已取得的上下文仍然可以使用。`task_persisted`、`receipt_persisted` 和 `memory_error` 分开记录，因此“没有记忆”和“存储出错”不会在回执中表现成同一种状态。

3.3 正确性约束下的运行时优化

性能优化遵守一个简单约束：注入模型的上下文和任务后的数据库状态必须保持逐字一致。在这个约束下，我依次完成了三步改造：把相同 revision 的编译结果移出编码热路径；把 TypeScript bridge 纳入正式 `tsdown` 构建；再用 `prepareRun` 在一个 Node 进程内完成快照读取、范围选择和任务写入。安装后的 CLI 执行预编译产物，源码检出没有构建产物时仍可回退到 `tsx`。

最终三臂实验包含 20 组配对。source/`tsx` 双调用、precompiled 双调用和 precompiled one-shot 的均值分别为 0.454、0.323、0.245 秒，p95 分别为 0.472、0.336、0.253 秒。最终路径把模型前桥接进程由 4 个降为 1 个，相对 source/`tsx` 双调用的均值和 p95 下降 46.1% 和 46.5%；20/20 组上下文和最终快照完全一致。该测量只覆盖本地 Python→Node→SQLite 固定开销，不包含模型、网络或 checker。

3.4 系统边界

- SQLite 单写者，CLI 文件锁只协调同一状态目录；没有多设备同步或后台归档。
- 默认最多 128 条观察；一次普通 `run` 通常使用任务和工具回执两条，容量满后仍可读旧上下文，但不能假装新任务已持久化。
- `scoped` 预算不足时回退完整原话而不静默裁切，因此上下文可能超过目标字节预算。
- `changes.diff` 相对当前 HEAD，可能包含用户已有编辑；它是审阅材料，不是模型归因证明。
- 当前交付没有自动接入 Gateway，也没有发布新的 npm 正式版本或实现自动语义纠错。
- 提示 JSON 采用无损紧凑序列化；在既有 6 份顺序任务 context 上字节数从 4,383 降到 4,247（3.10%）；这是确定性字节重算，不等同于 token 费用或模型质量实验。

4. 实现与测试代码清单

4.1 主要实现

路径	职责
src/core/memory-feedback/project-memory.ts	项目记忆状态机、校验、范围、版本链、撤销与上下文视图
src/core/memory-feedback/index.ts	明确导出的项目记忆 API 与类型
src/core/store/sqlite.ts、types.ts	原生 SQLite 精确记录能力与存储合同
scripts/project-agent/project_agent.py	remember/context/run/history/retract/check-run 编排
scripts/project-agent/backend.py	Codex/Claude CLI、实时 JSONL、usage、超时与取消
scripts/project-agent/changes.py	已跟踪和普通新文件的有界差异捕获
scripts/project-agent/store.ts	Python 到 MemoryCore SQLite 的 JSON 桥接
bin/memory-agent.mjs	安装后统一命令入口与运行时定位

4.2 主要测试

路径	覆盖重点
src/core/memory-feedback/project-memory.test.ts	来源校验、隔离、范围、更新链、损坏链、撤销、容量和 fallback
scripts/project-agent/test_project_agent.py	CLI 流程、回执、错误边界、恢复与读写失败语义
scripts/project-agent/test_backend.py	后端命令、实时日志、超时、取消、进程组终止和 usage
scripts/project-agent/test_changes.py	已有/新增文件差异、符号链接、容量和遗漏原因
benchmarks/topic3-be-agent-product-v1/test_*.py	runner 隔离合同、公共结果聚合、MemoryCode 评分和证据对齐
benchmarks/topic3-be-agent-product-v1/project_memory_runtime_benchmark.py	source/预编译/one-shot 三臂调用数、上下文与最终状态等价性、配对时延
scripts/ci/smoke-memory-agent-package.sh	从生成 tarball 安装后的入口、子路径导出及 remember→context→history
.github/workflows/pr-ci.yml	全量/最低版本测试、格式、构建、打包、尺寸、manifest 和隔离门禁

5. 测试方法

5.1 工程验证

工程测试覆盖六层：状态机、执行宿主、公共评测工具、完整仓库回归、最低支持版本和最终安装包。单元测试不依赖网络或真实模型；安装 smoke 则在空消费者目录中安装生成的 .tgz，专门检查“源码工作区能跑、发布包不能跑”这一类交付问题。PR 的每个新 head 都由 GitHub Actions 重跑相同门禁。

5.2 效果验证

效果实验固定任务、base commit、模型配置和 checker，各 arm 使用独立文件系统并禁用 web search，同时保存完整事件与 usage。顺序任务的第二步可能继承第一步代码，因此我把“独立项目家族的首个必要 任务”作为胜负单位，避免把一次修复重复计算成多次收益。

评测中最耗时、也最重要的工作，是判断一个分数是否真的由记忆带来。v5–v9 先后暴露未来 Git ref、兄弟 arm 路径、联网搜索、Git porcelain 解析和 checker 语义覆盖等问题。这五套矩阵全部保留用于复盘，但不进入最终成绩。只有通过隔离扫描、上游实现、独立等价实现和 near-miss 反例预检的 v4 与 v10 被 计入产品结论。最终样本量因此更小，但每一个胜例都能说明“历史提供了当前代码之外的必要决策”。

6. 测试结果

6.1 最终工程结果

检查	结果
MemoryCore Node 24 全量 Vitest	24 个文件，204/204 通过
Python 项目代理	25/25 通过
Python agent-product／公共评测 runner	31/31 通过
项目记忆 runtime 三臂配对	20/20 上下文、最终快照等价；模型调用 0
最低 Node 22.19 反馈模块	5 个文件，30/30 通过
Black 25.1.0+Prettier 3.5.3	通过
<code>npm run build</code>	通过
生成包	376 个文件，1,479,928 bytes，低于 2 MiB 门禁
空目录安装后的 CLI／SQLite／子路径 smoke	通过
Python 编译、shell 语法、 <code>git diff --check</code>	通过

终检还发现，早期公共评测夹具缺少现行回执协议要求的 `shard`、模型和解码字段。我补全了合法回执，并把公共评测套件纳入 CI，而没有放宽评分器。随后增加跨回执的模型、解码和 token 限额一致性检查；最终 31 项公共评测测试全部通过，原有 72 条模型回执的重算结果逐字段保持不变。这项修复没有改变实验 分数，但补齐了从原始回执到报告数字之间的验证链。

6.2 受控公开仓库编码任务

有效矩阵	独立项目家族	Full raw	No history	首任务家族胜负	严重回归
held-out v4	packaging、Flask、tqdm、Uvicorn	8/8	6/8	1 胜 0 负 3 平	0
held-out v10	h2、pycodestyle、path、importlib_resources	8/8	6/8	1 胜 0 负 3 平	0
合计（按独立家族首任务）	8 个家族	—	—	2 胜 0 负 6 平	0

两个有效收益都属于“当前代码和通用常识不能唯一确定维护者选择”的情况：tqdm 需要保留已知长度提示、忽略未知提示；importlib_resources 需要采用指定异常类型及诊断语义。完整原话给出了决定性历史，no-history 选择了合理但不被项目接受的另一实现。

两套矩阵中的成本都只测量一次，只能作为机制线索。v4 中 full raw 相比 no-history 的总输入、非缓存输入 和墙钟分别低 30.16%、17.06%、12.46%；v10 分别低 51.28%、57.01%、34.40%。合理解释是，明确的历史 约定减少了代理探索，而不是“上下文越多成本越低”。v4 的 raw top-8 虽然同样 8/8，并少注入 30.86% 上下文，却多用了 7.43% 总输入、23.14% 非缓存输入和 4.04% 墙钟，因此没有被采用为默认路径。

6.3 公共数据集与组件研究

下表汇总不同公开数据和组件实验。任务、模型与指标并不相同，所以分别报告，不合成为一个失真的总分。

数据／任务	量化结果	支持的结论
MemoryCode 全量检索	360 对话、4,182 查询；最新来源 recall@8 45.89%，100-session 为 20.01%；更新题 stale-only 24.12%	固定 top-8 在长历史和版本更新上会漏证据
MemoryCode 冻结生成	24 对话、72 调用；full history strict 12.50%，FTS 16.67%，1 胜 0 负 23 平；10 个更新题两者均 0%	小模型下普通检索没有高置信质量收益
MemoryCode Codex 完整更新层	固定全部 10 个更新 dialogue、20/20 调用有效；full raw 7/10，no history 0/10，即 7 胜 0 负 3 平；bootstrap 95% [+0.40,+1.00] ，sign p=0.015625	Codex 能从原话历史恢复多数更新约定；这是公开开发数据的方法诊断，不是产品成绩
MemoryCode focus 不重叠确认	新 10 个 update dialogue、与此前 34 个已用 dialogue 零重叠；raw 9/10，focus 10/10，1 胜 0 负 9 平；30/30 调用有效	修正后的两阶段整理得到方向性零观察回归；区间 [0,0.3] 、sign p=1.0，仍非高置信普适收益
MemoryCode source-context 不重叠确认	再新 10 个 update dialogue、与此前 44 个已用 dialogue 零重叠；raw 9/10、source 9/10，1 胜 1 负 8 平；热编码非缓存输入 0.693 倍	有明确降本线索，但未过零回归门槛，不启用
MemoryCode single-pass pilot	两个已暴露机制题 1 胜 1 负；active-rule 均分回退；非缓存输入 1.107 倍	在第三题和新留出前按协议停止
CUPID 纠正 ICL	相比普通历史 3 胜 4 负 5 平，输入 3.434 倍	当前规则／画像式反馈学习无稳定净收益
ValidMem 生命周期	374/406→387/406，15 胜 2 负；batch sign p=0.125	支持来源绑定与生命周期方法，不能当编码收益
Trigger 微任务	112/140→130/140，21 胜 3 负	能减少无关记忆操作；同源场景簇限制外推
STALE 候选提名	直接绑定和提议+裁决均 13/16；独立 candidate-only 15/16	只支持候选提名，不支持自动失效／持久化

MemoryCode 详细的已提交结果见 `topic3-be-agent-product-v1/MEMORYCODE_RESULTS.md`。完整更新层的 3 项冻结平局中，两项来自 attribute 提取器只识别字面 `self`：对应输出实际使用了 满足目标后缀的构造器属性，同时遵守另一条“重命名 receiver”的历史约定。接收者感知的 AST 事后 敏感性为 9/10，真实目标遗漏保留 1 项；因为输出可见后才加入该分析，所以 9/10 只作次级解释，不能 替换 7/10 主结论。full raw 相比 no history 的总输入、非缓存输入和墙钟分别增加 120.56%、411.37% 和 51.68%，明确暴露了直接注入完整历史的效率代价。

focus 的第一批开发集出现 1 个回归，原因是编译器错误地把 method 前缀、后缀、必含子串和数字约束 视为互斥；只修复该机制后才冻结第二批新集。第二批冻结 official-compatible active-rule 均分为 88.67%→91.78%；事后 receiver-aware 全规则敏感性为 89.44%→97.00%、3 胜 0 负 7 平，因该聚合在 输出后新增，只作次级诊断。逐请求冷 focus 为 2.011 倍输入、2.026 倍非缓存输入、1.706 倍墙钟和 2 倍调用；持久复用编译状态后的实际 coding stage 为 1.009 倍输入、1.101 倍非缓存输入、0.969 倍 墙钟和相同调用数。compact-history pilot 有目标回归，独立 CLI 的共享前缀 pilot 没有产生非缓存收益，两者均已关闭。再进一步的 source-context 在 54 个已用包上确定性减少 37.03% 历史字节，第三批不重叠 确认的热编码输入／非缓存输入为 raw 的 0.809／0.693 倍，active-rule 为 4 胜 0 负 6 平；但冻结主指标 为 1 胜 1 负 8 平，仍保持 off。single-pass 在两个机制题上 1 胜 1 负且成本门槛失败，未扩样。完整可重算结果位于研究分支 `delivery/topic3-be-product-v1`，当前冻结 head 为 `7e19d4f`。

有效公开仓库矩阵和 Codex 诊断的协议、回执哈希与原始证据保存在研究分支 `delivery/topic3-be-product-v1` 及本机 `.local-evidence/` 归档；产品交付分支通过冻结研究基线继承提交内 证据，但产品 PR 的增量审阅仍聚焦方案实现、测试与最终报告，不把本机生成产物加入 Git。

模型侧降本的整体失败门槛见 `MEMORYCODE_INFRA_OPTIMIZATION_RESULTS.md`；已采用的本地 bridge 等价性与时延结果见 `PROJECT_MEMORY_BRIDGE_RESULTS.md` 和 `PROJECT_MEMORY_RUNTIME_OPTIMIZATION_RESULTS.md`。

6.4 结论的适用范围

- 尚未选择并运行外部开源项目记忆方案的同任务、同模型对照。
- Claude Code 在早期服务诊断中连续返回 502，没有形成可评分结果；服务失败没有计作质量失败。
- Codex CLI 记录请求模型与 usage，但不能独立确认服务端实际模型身份或账户美元费用。
- 现有证据支持实验系统和窄范围收益，不支持总体成功率、商业可用性或普遍优于 Codex/Claude Code/ 开源系统的结论。

7. 复现工程测试

以下命令不调用模型：

```
cd MemoryCore
npm install --ignore-scripts --legacy-peer-deps
python3 -m pip install -r scripts/project-agent/requirements-dev.txt

npm run lint:project-agent
npm test
npm run test:project-agent
npm run test:agent-product
npm run build
npm pack
bash scripts/ci/smoke-memory-agent-package.sh ./*.tgz
```

最低 Node 验证应使用 Node 22.19.0 运行：

```
npx vitest run src/core/memory-feedback
```

真实模型实验不属于普通测试命令。复现时需要独立 worktree、已登录的 backend，以及冻结的 manifest 和 checker；smoke、旧工作区和已经打开过的任务不能作为新的留出实验。

8. 最终结论

本项目完成的不是一个孤立的检索算法，而是一套可以安装和审阅的项目记忆系统：它有明确 API、持久状态、来源与范围语义、故障回退、检查恢复、完整运行回执、最低版本验证和 CI 门禁。实现代码、测试代码、公共实验和本报告已经收束到同一 PR，能够由另一位开发者从安装包开始复核。

研究上的主要发现是：当任务涉及源码无法唯一决定的维护者选择时，完整原话项目记忆确实能改变编码结果。在 8 个公开项目家族上，它带来 2 个独立收益且没有首任务损失；在 MemoryCode 更新任务上，Codex 的完整历史基线达到 7/10，无历史为 0/10，修正后的约束整理又在独立确认集上达到 9/10→10/10。与此同时，6/8 项目家族仍然打平，统计区间也提醒我们不要把窄范围收益写成普适结论。

工程上的主要成果是把正确性、可审计性和开销放在同一条链路中优化。编译状态不再在每次编码时重算；预编译 one-shot bridge 将模型前进程由 4 个降到 1 个，本地均值和 p95 固定开销下降 46.1% 和 46.5%，而 20/20 组上下文与最终快照完全一致。基于这些证据，本项目可以表述为：完成了一套面向编码代理的可演化项目记忆方案，并在公开仓库与长对话数据上验证了其有效边界，同时交付了可安装、可测试、可回退、可审计的完整实现。外部开源系统的同条件对照与更大规模泛化验证留作后续工作。

9. 详细量化附录

本附录补充正文中不适合全部展开的协议、逐层指标和失效记录。不同数据集的分母、模型和任务不同，所有结果分别报告，不合成为一个失真的“总准确率”。

9.1 MemoryCode 数据与全量检索

- Source commit : [1ab87e119b2f9a498de8075219e1c07f6041b394](#)
- Hugging Face revision : [32d888b11c73c67be91414e571dfe98c5c20feac](#)
- 规模：360 dialogues、8,400 sessions、4,426 次规则新增、2,913 次规则更新、4,182 个查询
- 检索：每个 dialogue 独立 SQLite，原始 session chunk，FTS5，固定 k=8，最新 session 强制保留

Session 数	查询	最新来源 recall@8	更新查询	stale-only
1	43	100.00%	0	—
2	45	100.00%	0	—
3	66	96.97%	7	0.00%
4	79	98.73%	20	0.00%
5	108	96.30%	23	0.00%
10	152	89.47%	37	0.00%
15	244	72.95%	49	24.49%
20	299	70.57%	66	22.73%
30	424	55.66%	89	25.84%
40	545	44.40%	114	42.98%
50	813	38.01%	133	28.57%
100	1,364	20.01%	283	21.55%
总计	4,182	45.89%	821	24.12%

更新查询最新来源 recall 为 46.77%，旧版本碰撞率为 61.39%。检索 mean/p50/p95 为 1.546/1.387/2.785 ms；短历史 recall 87.92%，长历史为 36.89%。这说明普通 top-8 在长历史和版本更新上会漏证据，不能只靠增大 k 在已打开结果上追分。

9.2 MemoryCode 24-dialogue、72-call 三臂生成

模型为 Qwen3-4B-Instruct-2507，greedy，输入上限 65,536，输出上限 1,024；24 个 dialogue 覆盖全部 12 档历史长度，每档两个独立 cluster。

Arm	Strict target	Official-compatible	Coverage	Input	Gen p50/p95	截断
Full history	3/24 (12.50%)	23.99%	91.67%	304,702	11.80/41.34s	1
MemoryCore top-8	4/24 (16.67%)	19.94%	100.00%	70,706	7.26/16.05s	0
Latest-rule oracle	13/24 (54.17%)	65.85%	95.83%	5,701	9.09/24.67s	1

top-8 对 full history 为 1 胜 0 负 23 平，bootstrap 95% [0,0.125]，sign p=1.0；输入为 full history 的 23.20%，但 official-compatible 为 3 胜 7 负 14 平。10 个 update 任务两条普通路径 均为 0%，oracle 为 50%，所以后续工作转向来源绑定和版本整理。

9.3 Codex 更新层与 focus

全部 10 个 update dialogue 的 Codex 运行使用 CLI 0.153.4、请求 gpt-5.6-sol、medium、空只读目录、无工具与 web。20/20 调用有效。

指标	No history	Full raw	变化
Strict target	0/10	7/10	7 胜 0 负 3 平
Input	116,270	256,448	+120.56%
Non-cached input	37,934	193,984	+411.37%
Output	4,879	11,150	+128.53%
Wall	180.223s	273.357s	+51.68%

质量差均值 +0.70，bootstrap [+0.40,+1.00]，sign p=0.015625。receiver-aware 事后敏感性为 9/10，但冻结主结果保留 7/10。

机制修正后的第二批零重叠 focus 确认：30/30 调用有效，receiver-aware target 由 9/10 到 10/10，1 胜 0 负 9 平；active-rule mean 89.44%→97.00%，3 胜 0 负 7 平。主区间 [0,0.30]、sign p=1.0，只能作为方向性证据。

Focus 成本	Raw full	Cold focus	Persisted coding stage
Model calls	10	20	10
Non-cached input	189,747	384,398 (2.026x)	208,938 (1.101x)
Wall	360.637s	615.355s (1.706x)	349.318s (0.969x)

产品因此只采用“显式反馈写入时编译、精确 revision 持久复用”，不采用逐请求冷编译。

9.4 CUPID、ValidMem、Trigger、STALE

数据／任务	规模与结果	成本与边界
CUPID	756 observations、252 personas；固定 4 personas/12 tasks/48 calls；feedback 对 frozen 为 3W/4L/5T，cluster mean -0.0833，CI [-0.25, 0]	输入 3.434x、+73,260；time 1.620x；负结果使默认编译关闭
ValidMem v1.1	406-case 留出：374/406→387/406，15W/2L；Part B/EAR 147/176→159/176	input +0.59%，wall +14.01%；batch sign p=0.125，只作生命周期方法验证
Trigger Bench	140-case 留出：full pass 112/140→130/140，21W/3L；specificity 62.12%→89.39%	工具调用 -34.72%，input -7.76%，wall -3.56%；公开微任务结果
STALE direct	13/16，T2 1/4，负例 8/8	16 calls；太保守，关闭最终绑定
STALE proposer+judge	13/16，1W/1L/14T	48 calls、455,788 input、303.180s；无净收益
STALE candidate-only	15/16，T1/T2 4/4，负例 7/8，引用错误 0	16 calls、228,302 input、171.741s；只授权候选提名，不授权失效

另 200 行 STALE candidate-universe 检索中，原始新 session 用户文本 BM25 recall@1/5/8/16/32 为 67.5%/90.5%/96.0%/98.0%/100%，T2 recall@8 为 94%；归一化 `M_new` 只有 61%/53%。

9.5 公开项目逐矩阵结果

有效 v4：packaging、Flask、tqdm、Uvicorn；24/24 调用完成、隔离 24/24、严重回归 0。Full raw 与 top-8 均 8/8，no-history 6/8；按首任务是 1 胜 0 负 3 平。tqdm 的关键历史要求是“忽略未知长度提示、保留最小已知提示，只有全部未知时才用零”。

v4	No history	Full raw	Top-8
Total input	2,919,958	2,039,342	2,190,824
Non-cached input	243,734	202,158	248,936
Output	31,169	27,409	29,413
Wall	956.214s	837.074s	870.874s
Context	0	16,484B	11,398B

有效 v10：h2、pycodestyle、path、importlib_resources；16/16 调用与 checker 完成，42/42 preflight 符合预期，包含 18 个被拒绝 near-miss；隔离／额外路径／严重回归均 0。Full raw 8/8，no-history 6/8；按首任务是 1 胜 0 负 3 平。importlib_resources 的历史提供了源码无法唯一确定的 `TypeError` 和诊断语义。

v10	No history	Full raw	变化
Total input	1,074,156	523,336	-51.28%
Non-cached input	189,548	81,480	-57.01%
Output	22,474	11,534	-48.68%
Wall	668.636s	438.657s	-34.40%
Context	0	17,322B	—

合计按任务行为是 full raw 16/16、no-history 12/16；按独立项目家族首任务才是正式口径：2 胜 0 负 6 平。两项后续控制失败继承首任务错误，不能重复算成四个胜例。

9.6 失效矩阵记录

矩阵	发起／计划	原因	处理
v5	16/16	jsonschema no-history 两次 300s timeout, checker 未运行	缺失观察, 不记 memory win
v6	2/16	首臂 timeout, 配对臂人工取消	partial edit 不补作正式完成
v7	11/16	未强制 single-file edit scope	行为通过不能覆盖额外文件回归
v8	2/16	Git porcelain 前导空格被 <code>.strip()</code> 删除	runner 标签不可信, 立即停止
v9	16/16	typeguard checker 接受条件顺序 near-miss	整矩阵失效, 新增 near-miss 标准

五轮 event、receipt、partial workspace 和复盘保留, 但都不进入主成绩。这一部分是评测工作量的重要组成: 目标不是获得更多“胜利”, 而是确保保留下来的每个胜例能经得住代码和协议审阅。

9.7 工程、fallback 与恢复

检查	结果
MemoryCore Vitest	24 文件, 204/204
Python 项目代理	25/25
Python 公共评测	31/31
主测试合计	260/260
Node 22.19 memory-feedback	5 文件, 30/30
Runtime 三臂逐字等价	20/20, 模型调用 0
npm pack	376 文件, 1,479,928 bytes, 低于 2MiB
空目录安装 smoke	CLI、子路径导出、SQLite 读写通过

Runtime contract 四例全部通过: `feature_off` 返回 baseline; 正常选择 `k=1`; selector 强制失败返回 `selector_failed` 与原基线; `over-k` 返回 `invalid_selection` 与原基线。候选最大 256、`k` 1-32、timeout 最大 30s, adapter 不持有持久化写句柄。

容量恢复实跑: 状态预填到 128 observations/revision 128; 一次 Codex 调用仍取得 773-byte scoped context, 任务写入失败被标记为 `task_persisted=false`。编码 input 96,294 (cached 88,192)、output 2,213、wall 63.256s; checker 被主动取消后, `check-run` 以 0 次新模型调用补跑通过, 数据库仍保持 revision 128。

9.8 AI infra 最终结果

20 组轮换三臂	Bridge 进程	Mean	p50	p95
Source/tsx 双调用	2	0.454s	0.457s	0.472s
Precompiled 双调用	2	0.323s	0.329s	0.336s
Precompiled one-shot	1	0.245s	0.245s	0.253s

最终路径相对 source 双调用 mean/p95 降 46.1%/46.5%; 相对最初四进程结构, 进程数减少 75%。20/20 context 和最终 SQLite snapshot 逐字一致。该测量不包含模型、网络 and checker, 不是生产 SLO。

10. 实现细节与代码工作量

10.1 核心数据合同

对象	字段	约束
Observation	id/order/role/text	ID≤256, text≤32k, order 单调; role=user/tool
Constraint	key/quote/scope/sourceId/supersedes	单次≤4; quote≤4k 且在来源中唯一; 更新指向同 key/scope 当前前任
Scope	paths/actions	1-16 路径; read/edit/test/build/install; 拒绝绝对路径与 ..
Snapshot	owner/project/revision/arrays	默认容量 128, 可配但≤1,024; 规则≤容量×4
Context	text/selectedIds/status/reason	默认 12kB, 合法 1-64kB; off/selected/fallback

当前规则由 observation order、supersedes 与 retraction 决定。旧规则只以 historical 前任证据出现，不会重新激活。损坏链、存储降级或预算无法完整容纳时不返回部分 selected ID。

10.2 执行与证据顺序

```
校验 workspace/path/checker argv
→ 从同一 pre-task revision 读取 snapshot 与 context
→ 原子写 task.json, 尝试 ingest 当前任务
→ 启动 Codex/Claude 进程组, 实时写 stdout.jsonl/stderr
→ 原子写 agent-result.json 与 usage
→ 捕获相对 HEAD 的 changes.diff
→ 运行外置 checker 并保存原始输出
→ 写工具回执和最终 receipt
```

task_persisted、receipt_persisted、memory_error 分开记录。读取成功但任务写入失败时继续使用已取得 context；读取失败则退回当前请求。check-run 只恢复检查，不重复模型调用和记忆写入。

10.3 代码规模

从原始 MemoryCore 基线 0dde892f1e362b4b937b2b38d23beb2a5ac4329 到实现快照 84ba04d：166 个文件，+21,878/-83 行。B+E runtime 与项目代理主要实现 10 文件、2,368 行；聚焦测试 15 文件、2,158 行；公开评测、数据准备和 checker 源码 50 文件、8,973 行。最终产品化 PR 相对研究分支为 41 文件、+3,855/-1,088 行。

11. 结论边界与最终判断

可以确认：完整原话是当前最可靠的证据保留基线；普通 top-8 在长历史更新上会漏掉最新来源；强模型能够利用版本历史；范围、版本链、撤销、fallback 和执行证据已经形成可运行系统；项目历史在两个不同公开项目中解决了源码无法唯一确定的维护选择；模型前固定开销已经在等价条件下降低约 46%。

不能确认：普遍优于 Codex、Claude Code 或开源记忆系统；公开结果等同内部真实业务；候选提名可以自动失效；编译规则已获得稳定反馈学习收益；单次成本下降可直接外推为生产 SLO。Claude Code 诊断连续 502，没有质量样本，服务失败未计作质量失败。

最终结论是：本项目完成了一套面向编码代理的可演化项目记忆方案，并在公开长对话和公开仓库任务上验证了有效机制、失败边界和工程可交付性。它已经适合作为实验版和可审阅研究交付；扩大默认启用前，仍需更大规模新项目、同任务开源方案对照以及内部多轮编程数据复核。

12. 复现与移植

零模型复现命令：

```
cd MemoryCore
npm install --ignore-scripts --legacy-peer-deps
python3 -m pip install -r scripts/project-agent/requirements-dev.txt
npm run lint:project-agent
npm test
npm run test:project-agent
npm run test:agent-product
npm run build
npm pack
bash scripts/ci/smoke-memory-agent-package.sh ./*.tgz
```

内部数据只需替换 adapter、标签和 checker，主状态机与运行回执保持不变。内部记录至少需要 owner/project/ session/ turn、 user/tool role、 exact text、 repo/branch/environment、 path/symbol/action、 source/supersedes/retract、 base commit/diff、 backend/model/usage 和外置 checker。公开数据中的“最新值全局有效”不能直接移植到多分支环境。

13. 参考资料

1. TencentCloud, TencentDB Agent Memory / MemoryCore v2.0.0-beta.1.
2. Shinn et al., Reflexion: Language Agents with Verbal Reinforcement Learning, 2023.
3. Maharana et al., LoCoMo, ACL 2024.
4. Wu et al., LongMemEval, 2024.
5. Mem0, Building Production-Ready AI Agents with Scalable Long-Term Memory, 2025.
6. Rasmussen et al., Zep: A Temporal Knowledge Graph Architecture for Agent Memory, 2025.
7. Cohere Labs Community, MemoryCode.
8. Zhou et al., ValidMem / MemFSM.
9. Agent Memory Trigger Bench.
10. STALE: A Benchmark for Stale Memory in Long-Context Agents.

本文所有“通过”均对应明确协议和分母。开发结果、失效矩阵、事后敏感性和主结果分开标记；未用未完成 调用补数，也未把外部论文成绩记作本项目成绩。